

Management Information Systems

13. Quality Assurance

Guo, Xunhua

guoxh@tsinghua.edu.cn



清华经管学院
Tsinghua SEM

Quality Assurance

- Designing Accurate Data Entry Procedures (Chapter 15, Textbook)
 - **Effective Coding (数据编码)**
 - Effective and Efficient Data Capture
 - Ensuring Data Quality through Input Validation
- Quality Assurance and Implementation (Chapter 16, Textbook)
 - Total Quality Management
 - **Taming Complexity: Modularity & Documentation**
 - Deployment, Training, and Conversion
 - Security, Recovery, and Evaluation

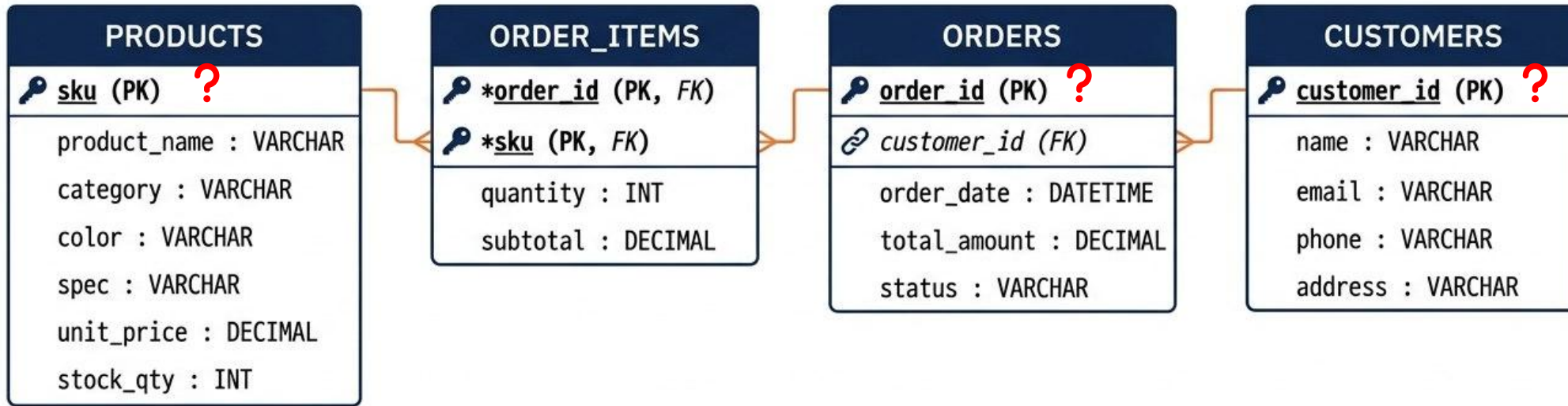


Designing Accurate Data Entry Procedures

- Effective Coding (数据编码)
- Effective and Efficient Data Capture
- Ensuring Data Quality through Input Validation



The Coding Decision



- Surrogate key (代理键，无业务意义编码)
 - Simple sequence code
- Meaningful codes (有业务意义的编码)
 - Block Sequence Code (分类序列编码)
 - Alphabetic derivation code (字母派生编码)
 - Significant-digit subset code (子集代号编码)



Effective Coding

- Data that are coded require less time to enter
- Coding helps to reduce the number of items entered
- Coding can help in sorting data during the data transformation process
- Coded data can save valuable memory and storage space



Human Purposes for Coding

- Keeps track of something
- Classifies information
- Conceals information
- Reveals information
- Requests appropriate action



Simple Sequence Code

- A number that is assigned to something if it needs to be numbered
- No relation to the data itself

Order #	Product	Customer
5676	Rocking Chair/with Leather	Arthur Hook Jr.
5677	Dining Room Chair/Upholstered	Millie Monice
5678	Love Seat/Upholstered	J. & D. Pare
5679	Child's Rocking Chair/Decals	Lucinda Morely



Block Sequence Code

- An extension of the sequence code
- Data are grouped according to common characteristics
- Simplicity of assigning the next available number (within the block) to the next item needing identification

Code	Name of Software Package	Type
100	Apple Safari	Browser
101	Mozilla Firefox	
102	Microsoft Internet Explorer	
103	Google Chrome	
.		
.		
200	Microsoft Access	Database
201	MySQL	
202	Oracle	
.		
.		
.		
300	Adobe Dreamweaver	Web design
301	Freeway Pro	
302	BlueGriffon	



Alphabetic Derivation Code

- A commonly used approach in identifying an account number

Code	Explanation of Code
68506KND7533TVG	99999XXX9999XXX Abbreviation of magazine Four digits of street address First three consonants in last name First five digits of zip code

Significant-Digit Subset Code

- Used to help describe a product by virtue of its membership in many subgroups
- Possible to locate items that belong to a certain group or class
- Inquiries may be performed on a portion of the code

Code	Merchandise Described	Explanation of Code			
2023954010	Red maternity dress, style 395, size 10	202	—	395	— 40 — 10
		↑ Department (Maternity)		↑ Product (Dress Style 395)	↑ Color (Red) ↑ Size (Size 10)
4142191912	Beige winter coat, style 219, size 12	414	—	219	— 19 — 12
		↑ Department (Winter Coats)		↑ Product (Coat Style 219)	↑ Color (Beige) ↑ Size (Size 12)



Other Codes

- Classification Code (分类码)
- Cipher Code (密码)
- Mnemonic Code (助记码)
- Function Code (功能码)

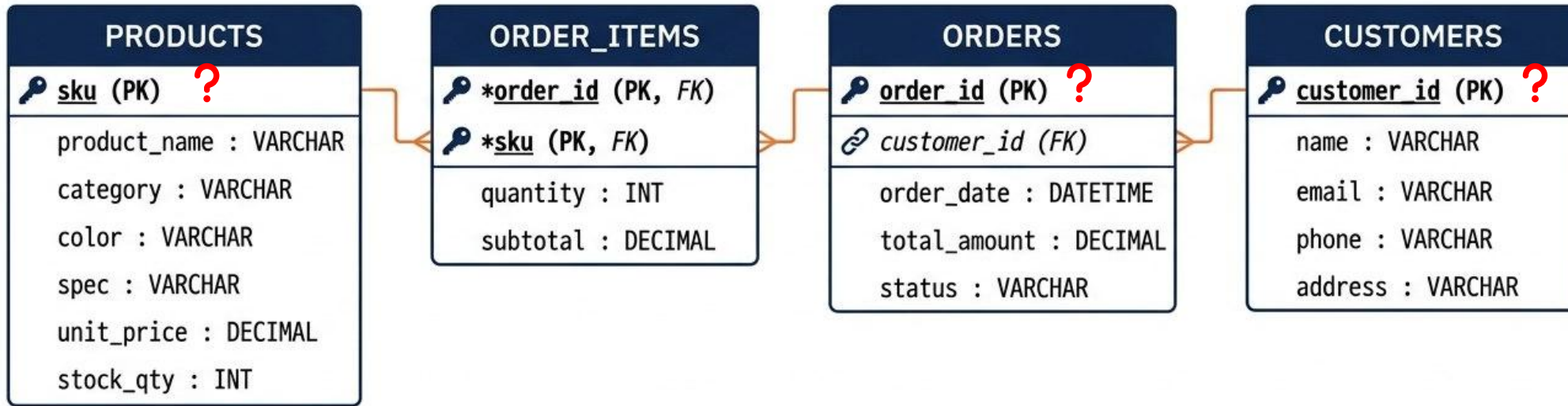


General Guidelines for Coding

- Be concise
- Keep the codes stable
- Ensure that codes are unique
- Allow codes to be sortable
- Avoid confusing codes
- Keep the codes uniform
- Allow for modification of codes
- Make codes meaningful



The Coding Decision



- sku: significant-digit subset
- customer_id: simple sequence
- order_id: block sequence

sku: Significant-Digit Subset Code

- Because products are often operated on by classification.
- For example
 - [category]-[model]-[color]-[specification]
 - LM-X3-W-STD
- "All speakers get 20% off for Double 11."

```
UPDATE price SET discount = 0.8 WHERE sku LIKE 'LM%';
```

- Without the code:

```
UPDATE price SET discount = 0.8  
WHERE sku IN (SELECT sku FROM product_category_map WHERE category = 'Audio');
```



Significant-Digit Subset Code: the Trade-off

- e.g.,
 - [category]-[model]-[color]-[specification]
 - LM-X3-W-STD

Gain	Cost
Human-readable at a glance	Segment meanings can go stale — if "LM" is renamed to "AU", every SKU must change
Sortable by any segment	Space limits per segment — only $26^2=676$ two-letter combinations
Bulk operations in one line of SQL	Violates "Keep Codes Stable" — the richer the semantics, the harder to modify



customer_id: Simple Sequence Code

- Because a person's identifier should carry no story.
 - e.g., 00000847
 - Display as C-00000847 when needed. Store as a bare integer.
 - No segments. No semantics. No story. Just identity.
- "A customer requests account deletion."

```
-- With Simple Sequence (surrogate key):
UPDATE customers
  SET name = '[DELETED]', phone = NULL, email = NULL
  WHERE customer_id = 00000847;
-- ✓ PII erased. Orders preserved for audit. Key never changes.

-- With phone number as PK:
DELETE FROM customers WHERE phone = '138xxxx5678';
-- ✗ Cascades through 87 tables – orders, payments, logistics, reviews...
-- Financial records break. Audit trail destroyed. System integrity fails.
```



Simple Sequence Code: the Trade-off

- e.g., 00000847
- Display as C-00000847 when needed. Store as a bare integer.
- No segments. No semantics. No story. Just identity.

Gain	Cost
Stable forever — nothing to go stale	Not human-readable — you need a lookup to know who 00000847 is
Privacy-safe — the key leaks nothing	Exposes scale — competitor can estimate your customer count
Deletable personal information without breaking references	Requires a separate display name/number for customer-facing contexts



order_id: Block Sequence Code

- Because orders are events and may be classified
- e.g.,
 - [type]-[date]-[sequence]
 - ORD-20260328-001847
- "Pull all ordinary orders from May 31, 2026 for the operations review."

```
SELECT * FROM orders  
WHERE order_id BETWEEN 'ORD-20260531-000000' AND 'ORD-20260531-999999';
```



Block Sequence Code

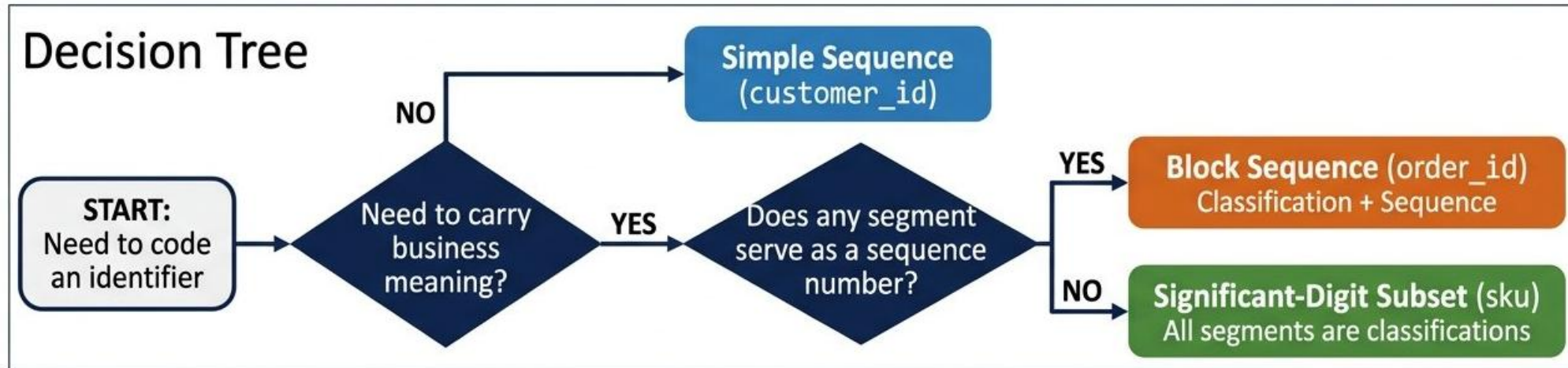
- e.g.,
 - [type]-[date]-[sequence]
 - ORD-20260328-001847

Gain	Cost
Naturally time-ordered — index stays clean	Within a day, counter is centralized — sharding needs care
Queries by date range = PK range scan	Reveals approximate daily volume to anyone who sees two IDs from the same day
Operators recognize timestamps at a glance	Slightly longer than pure counters (18 chars vs 8)
Trivial table partitioning by month/year	Cross-day operations still need a date index



The Coding Decision: Summary

- sku: significant-digit subset
- customer_id: simple sequence
- order_id: block sequence



Effective and Efficient Data Capture

- The Best Data Entry is *No Data Entry*
- The Core Principle
 - Enter what only humans can know; let the computer figure out the rest.
 - A base data item should be entered only once at only one point.

Question	What to Enter	What NOT to Enter
Is it changeable?	YES → Must capture each time (quantity, date, customer choice)	NO → Look it up from master data
Does it differentiate this record?	YES → Capture (the SKU being ordered)	NO → Derive it (the SKU's name, price, weight)
Can the computer compute it?	NO → Capture	YES → Generate it (timestamp, ID, total)



Example: an Order Form

✗ Bad Form — 5 human-typed fields

Customer Name:

← *re-typed every order*

Customer Phone:

← *already logged in!*

Product Name:

← *type "Lumina-X3 Speaker"?!*

Unit Price:

← *customer doesn't know this*

Total Amount:

← *customer calculates?*

✓ Good Form — 1 human-typed field

Customer: [Auto-filled from login]

Product: [search sku] → system fills name, price, weight...

Quantity:

← **the ONLY field a human enters**

Total: [Auto-calculated]

- Emerging data-entry methods

- OCR: auto-fill address; Computer vision: shelf recognition; LLM: extract data from unstructured emails; IoT sensor: auto-report stock level;



Ensuring Data Quality through Input Validation

- Bad data, once inside, is expensive to fix. Catch it at the door.
- Eight types of validation

1. Missing Data

Is a required field left empty?
e.g., customer_id must not be NULL

2. Field Length

Is the value the expected length?
e.g., Zip code = exactly 6 digits

3. Class / Composition

Is it the right character type?
e.g., Name field: letters only, no digits

4. Range / Reasonableness

Is the value within a plausible range?
e.g., Order quantity: 1–10,000; month: 1–12

5. Invalid Values

Is the value on a blacklist?
e.g., Order status must be P/S/D, not 'X'

6. Comparison with Stored Data

Does the foreign key exist in master data?
e.g., SKU must exist in Products table

7. Self-Checking Digits

Does the embedded check digit verify? (Luhn)
e.g., Credit card: last digit validates the rest

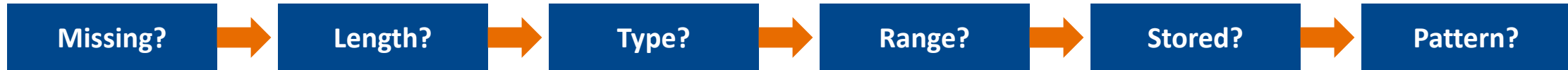
8. Pattern Match (Regex)

Does the value match a format pattern?
e.g., Email: x@x.x | Phone: (ddd) ddd-dddd



Validation: A Pipeline, Not a Checkpoint

The Validation Pipeline



✗ **FAIL** at any stage → reject + explain to user

✓ **All PASS** → accept into database

Two Layers of Defense

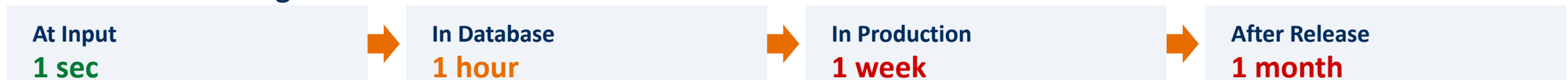
Client-Side Validation (Browser / App)

- Instant feedback — user sees errors before submit
- Catches honest mistakes (typos, wrong format)
- Improves UX — no frustrating round-trips
- ⚠ Can be bypassed (dev tools, API calls)

Server-Side Validation (The Final Defense)

- Never trust client data — re-validate everything
- Enforces business rules (range, stored data, check digits)
- Protects against malicious input (SQL injection, etc.)
- ☒ The only validation that truly matters

The Cost of Catching an Error



Quality Assurance and Implementation

- Total Quality Management
- **Taming Complexity: Modularity & Documentation**
- Deployment, Training, and Conversion
- Security, Recovery, and Evaluation



Total Quality Management (TQM)

- TQM
 - Full organizational support of quality management
 - Early commitment to quality from the analyst and business users
- The “Test-First” Principle (“测试先行”原则)
 - Test plan: All preparations for series of tests
 - Unit testing: Tests each program in system separately
 - System testing: Test functioning of system as a whole
 - Acceptance testing: Makes sure system is ready to be used in production setting
- Structured Walkthroughs (结构化复核)
 - Use peer reviewers to monitor the system’s programming and overall development
 - Point out problems
 - Allow the programmer or analyst to make suitable changes



Taming Complexity: Modularity & Documentation

- Divide and conquer — the oldest engineering principle, still the best.

Why Must Complex Systems Be Modular?

Human Cognition

No one can hold 100,000 lines of code in their head. Modules let each person understand one piece at a time.

Parallel Work

Teams work on different modules simultaneously — without stepping on each other's toes.

Change Isolation

Fix or replace one module without touching the rest. Change is local, not systemic.

The Top-Down Approach



Start from the whole → break into subsystems → break into modules → each module does ONE thing



Guidelines for Modular Programming

- Keep each module to a manageable size
- Pay particular attention to the critical interfaces
- Minimize the number of modules the user must modify when making changes
- Maintain the hierarchical relationships set up in the top-down phases

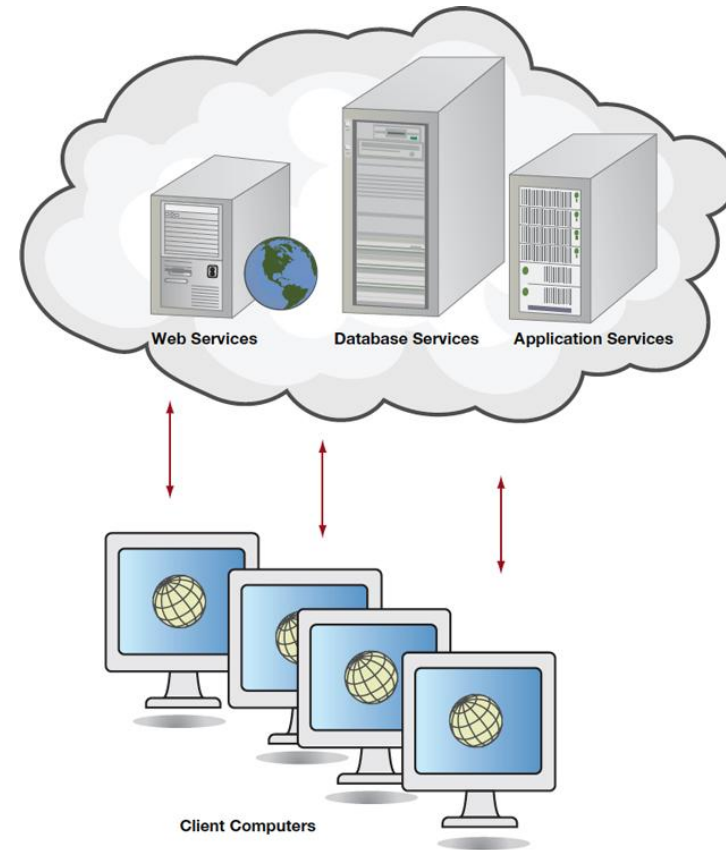
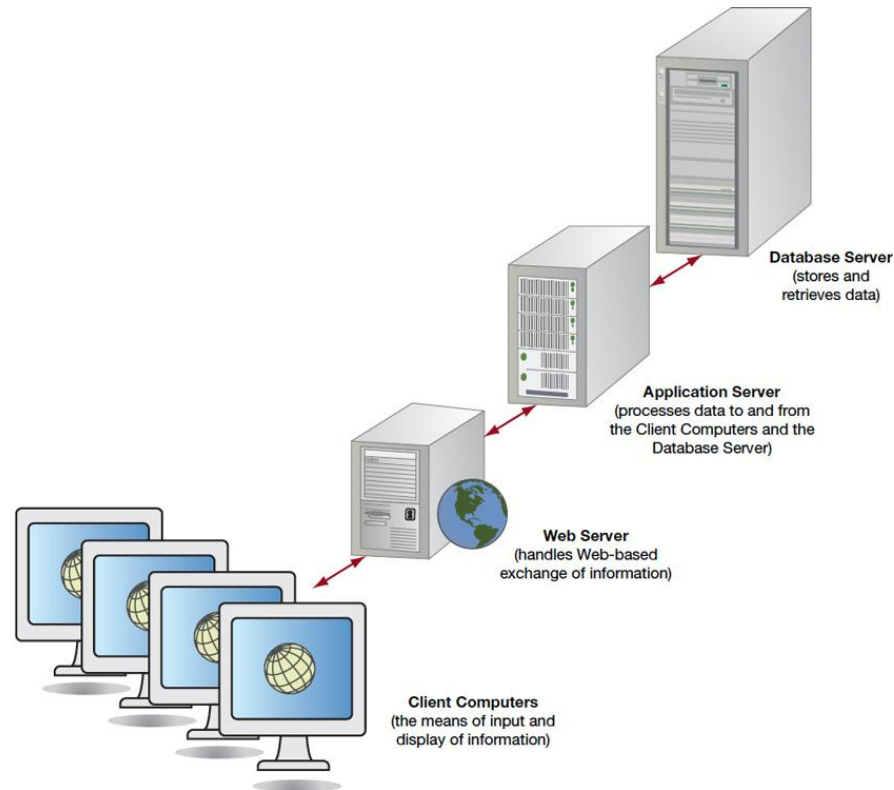


Documentations

- “If It Isn't Written Down, It Doesn't Exist”
- Procedure Manuals (the classic approach)
- The FOLKLORE Method (our textbook)
- 2026 Extension: **Docs-as-Code** + **AI-Assisted Documentation**
- Docs-as-Code
 - Documentation lives in Git, alongside code
 - Versioned, reviewed, and deployed with code
 - Tools: MkDocs, Docusaurus, Confluence+Git
- AI-Assisted Documentation
 - LLMs auto-generate docs from code
 - AI summarizes meeting notes into specs
 - Chatbots serve as "living documentation"



Deployment: Multi-tiered and Cloud Computing



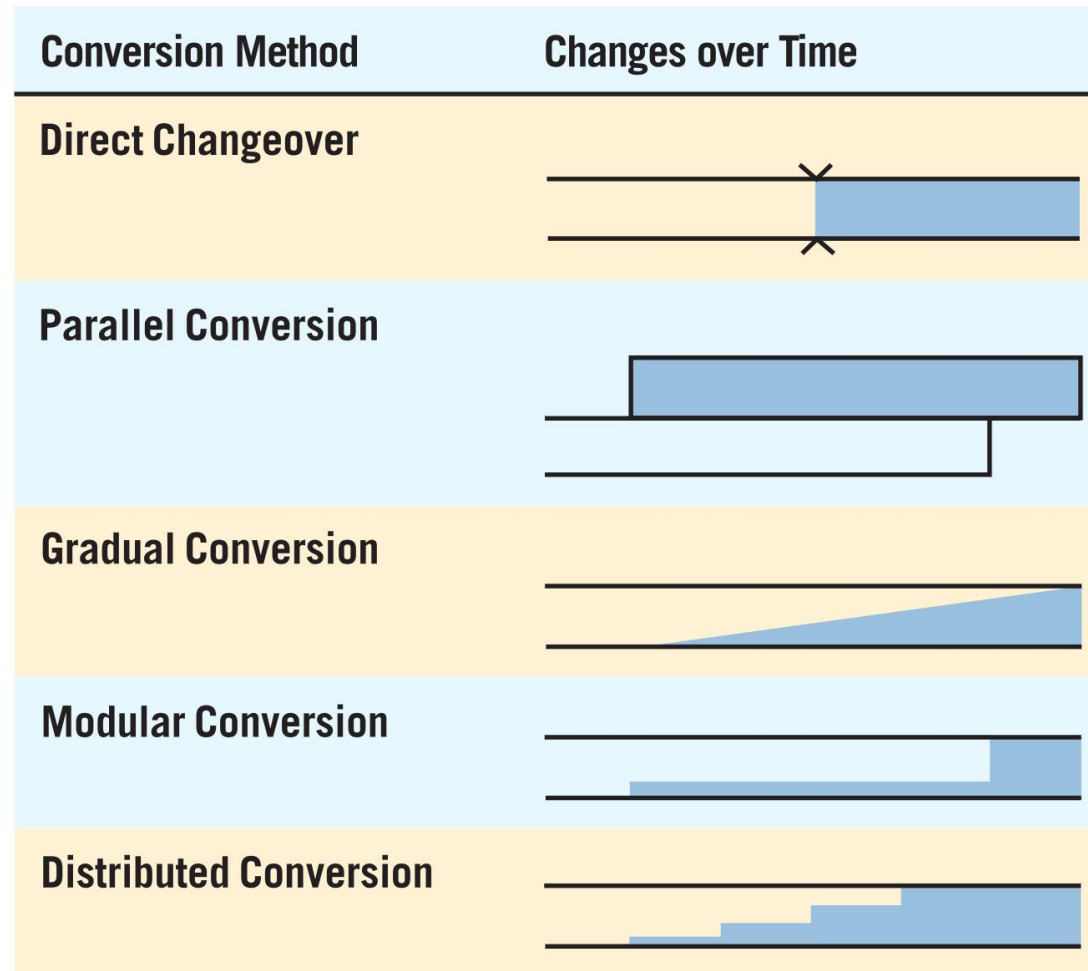
Training

Elements	Relevant Factors
Training Objectives	Depend on requirements of user's job
Training Methods	Depend on user's job, personality, background, and experience; use combination of lecture, demonstration, hands-on, and study
Training Sites	Depend on training objectives, cost, availability; free vendor sites with operable equipment; in-house installation; rented facilities
Training Materials	Depend on user's needs; operating manuals, cases, prototypes of equipments and output; online tutorials

- Training is not knowledge transfer — it's change management.
- Think about [Laowang](#) at Lumina



Conversion Strategies



Security, Recovery, and Evaluation

- Security
 - Authentication and authorization (权限控制), availability (可用性), and confidentiality (保密性)
- Disaster Recovery
 - Backup and redundancy: assume everything will fail eventually
- Evaluation
 - Define what "success" means before you build, or you'll never be able to prove it after.



Tasks

- Individual tasks
 - Reading: Textbook, Chapters 15-16
 - Answer the review questions (on YuKeTang)
 - Answer the additional review question (on WebLearning)
- Group tasks
 - Design your databases, codes, and data capture policies, and refine your prototype systems.
 - No submission is required for this week.
 - Final report submission is due **June 12**.
- Class presentations
 - June 8: Group 7 (蒋宇轩组, 智云科技) and Group 8 (胡布阳组, 卢米纳电子)
 - Content: all your progress with solution design and the prototype system.
 - Time: 10~20 minutes

